

SERVER SIDE MODULE

Contents

1. MODULE_SERVER_SIDE.docx
2. MODULE_SERVER_SIDE.pdf
3. MODULE_SERVER_SIDE_MEDIA.zip

Introduction

Our company “TechMart Solutions” appoints you as a Web Developer to build a web-based product management application using React as the frontend and Laravel 12 REST API as the backend. The application allows authenticated users to manage product data through features such as creating, viewing, up dating, and deleting products (CRUD).

User authentication is handled using Laravel Sanctum, ensuring that only authorized users can access and manage the data securely. The system stores all data in a MySQL database consisting of multiple related tables, including users, categories, products, and transactions. This application is designed to help users efficiently manage product information in a simple, secure, and scalable web application.

Description of Projects

There are 2 phases in this module. In the first phase, you will make the RESTFul API using Laravel Frameworks according to the provided documentation. In the second phase, you should build a frontend with the Interactive Maps using one of the provided Javascript Frameworks (React/Vue) and the data must come from the created RESTFul API.

You can find the provided media files to support your work:

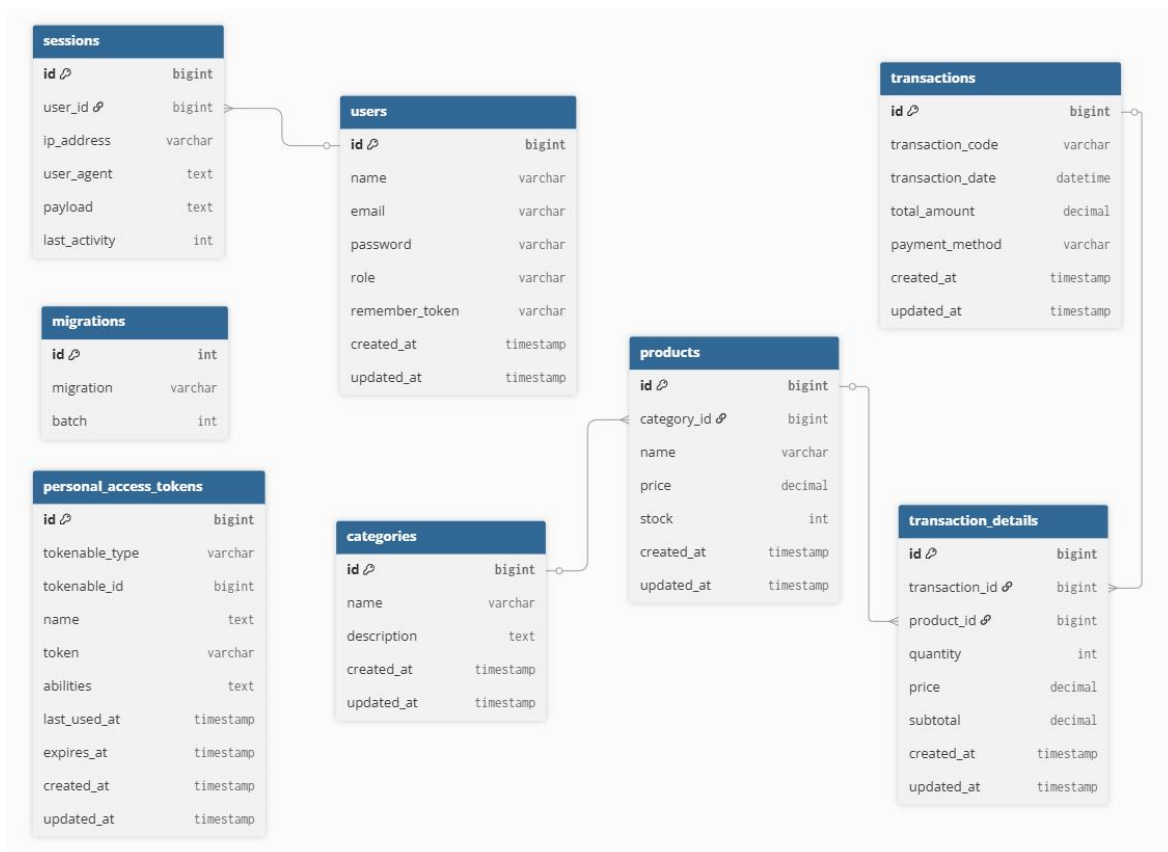
- Postman collection and environment
- Template GUI (to build frontend UI)
- database.sql (a database with structures)

Phase 1 : Database Preparation and Model Creation

In this phase, you are required to prepare the database structure based on the provided ERD (Entity Relationship Diagram).

- ✓ Set up the database configuration in the .env file.
- ✓ Carefully analyze the entities, attributes, primary keys, foreign keys, and relationships shown in the ERD.
- ✓ Generate Laravel models corresponding to each entity in the ERD.
- ✓ Run migration, ensure all tables are successfully created without errors.

Entity Relationship Diagram



Phase 2 : RESTful API

In this phase, you should build a RESTful API using the Laravel framework according to the documentation below.

You are required to insert predefined user data into the database using Laravel Seeder:

Name	Email	Password
User 1	user1@techmart.id	password1
User 2	user2@techmart.id	password2
User 3	user3@techmart.id	password3

1. Authentication

You should create Login and Logout endpoints. The **accessToken** must be generated by sanctum and will be placed in the request headers **Authorization Bearer**.

Login

Endpoint : [DOMAIN]/api/login

Description : For user to log in the system

Method : POST

Request : Body (JSON):

```
{
  "email": "user1@techmart.id",
  "password": "password1"
}
```

Validation :

- email:
 - required
 - valid email address
- password:
 - required
 - min 5 chars

Response :

If success:

Headers:

- Response status: 200

Body (JSON):

```
{
  "success": true,
  "message": "Login success",
  "user": {
    "name": "User 1",
    "email": "user1@webtech.id",
    "accessToken":
"1|QnHFgF0C7ml2LkbDG5QDn34qvJoYGdpxoXy63Poi"
  }
}
```

If invalid field:

Headers:

- Response status: 422

Body (JSON):

```
{
  "success": false,
  "message": "Invalid field",
  "errors": {
    "email": [
      "The email must be a valid email address."
    ],
    "password": [
      "The password field is required."
    ]
  }
}
```

If failed:

Headers:

- Response status: 401

Body (JSON):

```
{
  "success": false,
  "message": "Email or password incorrect"
}
```

Logout

Endpoint : [DOMAIN]/api/logout

Description : For user to log out the system

Method : POST

Request : Headers:

- Authorization: "Bearer <accessToken>"

Response : **If success:**

Headers:

- Response status: 200

Body (JSON):

```
{
  "success": true,
  "message": "Logout success"
}
```

If invalid token:

Headers:

- Response status: 401

Body (JSON):

```
{
  "success": false,
  "message": "Unauthenticated."
}
```

2. Categories

In this task, participants are required to develop a RESTful API for managing Categories data following the standard CRUD (Create, Read, Update, Delete) operations.

Create a Category

```
Endpoint      : [DOMAIN]/api/categories

Description   : For user to create new category

Method        : POST

Request       : Headers:
                - Authorization: "Bearer <accessToken>"

Validation    : name:
                - required
                description:
                - nullable

Response      : If success:
                Headers:
                - Response status: 200

                Body (JSON):
                {
                    "success": true,
                    "message": "Create form success",
                    "data": {
                        "id": 1,
                        "name": "Elektronik",
                        "description": "Produk elektronik seperti HP dan Laptop"
                    }
                }

                If invalid field:
                Headers:
                - Response status: 422

                Body (JSON):
                {
                    "success": false,
                    "message": "Invalid field",
                    "errors": {
                        "name": [
                            "The name field is required."
                        ]
                    }
                }
            }
```

```
If invalid token:
Headers:
  - Response status: 401

Body (JSON)
{
  "success": false,
  "message": "Unauthenticated."
}
```

Get all Categories

```
Endpoint      : [DOMAIN]/api/categories
Description   : For users to get all of their created categories
Method        : GET
Request       : Headers:
                - Authorization: "Bearer <accessToken>"
Response      : If success:
                Headers:
                  - Response status: 200

                Body (JSON):
                {
                  "success": true,
                  "message": "List of categories",
                  "data": [
                    {
                      "id": 1,
                      "name": "Elektronik",
                      "description": "Produk elektronik seperti HP dan
Laptop",
                      "created_at": "2026-02-11T10:00:00.000000Z",
                      "updated_at": "2026-02-11T10:00:00.000000Z"
                    },
                    {
                      "id": 2,
                      "name": "Makanan",
                      "description": "Produk makanan dan minuman",
                      "created_at": "2026-02-11T10:00:00.000000Z",
                      "updated_at": "2026-02-11T10:00:00.000000Z"
                    }
                  ]
                }
```

If invalid token:

Headers:

- Response status: 401

Body (JSON):

```
{
  "success": false,
  "message": "Unauthenticated."
}
```

Get Category by ID

Endpoint : [DOMAIN]/api/categories/<id>

Description : For users to get category by ID

Method : GET

Request : Headers:

Authorization: "Bearer <accessToken>"

Response : **If success:**

Headers:

- Response status: 200

Body (JSON):

```
{
  "success": true,
  "message": "Get category by ID",
  "data": [
    {
      "id": 1,
      "name": "Elektronik",
      "description": "Produk elektronik seperti HP dan Laptop",
      "created_at": "2026-02-11T10:00:00.000000Z",
      "updated_at": "2026-02-11T10:00:00.000000Z"
    }
  ]
}
```

If not found:

Headers:

- Response status: 404

Body (JSON):

```
{
  "success": false,
  "message": "Product not found",
}
```

If invalid token:

Headers:

- Response status: 401

Body (JSON):

```
{
  "success": false,
  "message": "Unauthenticated."
}
```

Update Category

Endpoint : [DOMAIN]/api/categories/<id>

Description : For users to update category

Method : PATCH

Request : Headers:
Authorization: "Bearer <accessToken>"

Response : **If success:**

Headers:

- Response status: 200

Body (JSON):

```
{
  "success": true,
  "message": "Category updated successfully",
  "data": {
    "id": 1,
    "name": "Updated Electronics",
    "description": "Updated electronic products category",
    "created_at": "2026-02-10T10:00:00.000000Z",
    "updated_at": "2026-02-11T08:30:00.000000Z"
  }
}
```

If not found:

Headers:

- Response status: 404

Body (JSON):

```
{
  "success": false,
  "message": "Category not found",
}
```

If invalid token:

Headers:

- Response status: 401

Body (JSON):

```
{  
  "success": false,  
  "message": "Unauthenticated."  
}
```

Delete Category

Endpoint : [DOMAIN]/api/categories/<id>

Description : For users to delete category

Method : DELETE

Request : Headers:

Authorization: "Bearer <accessToken>"

Response : **If success:**

Headers:

- Response status: 200

Body (JSON):

```
{  
  "success": true,  
  "message": "Category deleted successfully"  
}
```

If not found:

Headers:

- Response status: 404

Body (JSON):

```
{  
  "success": false,  
  "message": "Category not found",  
}
```

If invalid token:

Headers:

- Response status: 401

Body (JSON):

```
{  
  "success": false,  
  "message": "Unauthenticated."  
}
```

3. Products

In this task, participants are required to develop a RESTful API for managing Products data following the standard CRUD (Create, Read, Update, Delete) operations.

Method	Endpoint	Auth	Description
GET	[DOMAIN]/api/products	Yes	Get all products
GET	[DOMAIN]/api/products/{id}	Yes	Get a specific product
POST	[DOMAIN]/api/products	Yes	Create a new product
PATCH	[DOMAIN]/api/products/{id}	Yes	Update product
DELETE	[DOMAIN]/api/products/{id}	Yes	Delete a product

The JSON response format must follow the same structure as the Category API.
Example: List of Products (GET /api/products)

```
Endpoint      : [DOMAIN]/api/products

Description   : For users to get all of their created products

Method        : GET

Request       : Headers:
                - Authorization: "Bearer <accessToken>"

Response      : If success:
                Headers:
                - Response status: 200

                Body (JSON):
                {
                  "success": true,
                  "message": "List of products",
                  "data": [
                    {
                      "id": 1,
                      "name": "Laptop Asus ROG",
                      "description": "Laptop gaming 16GB RAM",
                      "price": "15000000.00",
                      "stock": 5,
                      "category": {
                        "id": 1,
                        "name": "Elektronik"
                      }
                    },
                    {
                      "id": 3,
                      "name": "Snack Kentang",
                      "description": "Snack rasa original 100gr",
```

```

        "price": "10000.00",
        "stock": 100,
        "category": {
            "id": 2,
            "name": "Makanan"
        }
    }
]
}

If invalid token:
Headers:
    - Response status: 401

Body (JSON):
{
    "success": false,
    "message": "Unauthenticated."
}

```

4. Transactions

In this task, participants are required to develop a RESTful API for managing Transactions data following the standard CRUD (Create, Read, Update, Delete) operations.

Method	Endpoint	Auth	Description
GET	[DOMAIN]/api/transactions	Yes	Get all transactions
GET	[DOMAIN]/api/transactions/{id}	Yes	Get a specific transactions
POST	[DOMAIN]/api/transactions	Yes	Create a new transactions
PATCH	[DOMAIN]/api/transactions/{id}	Yes	Update transactions
DELETE	[DOMAIN]/api/transactions/{id}	Yes	Delete a transactions

The JSON response format must follow the same structure as the Category API.

Example: List of Products (GET /api/transactions)

```

Endpoint      : [DOMAIN]/api/transactions

Description   : For users to get all of their created transactions

Method       : GET

Request      : Headers:
               - Authorization: "Bearer <accessToken>"

```

Response : **If success:**

Headers:

- Response status: 200

Body (JSON):

```
{
  "success": true,
  "message": "List of transactions",
  "data": [
    {
      "id": 1,
      "transaction_code": "TRX-20260211-001",
      "transaction_date": "2026-02-11 10:15:00",
      "total_amount": "15020000.00",
      "payment_method": "Cash"
    },
    {
      "id": 2,
      "transaction_code": "TRX-20260211-002",
      "transaction_date": "2026-02-11 11:00:00",
      "total_amount": "170000.00",
      "payment_method": "Transfer"
    }
  ]
}
```

If invalid token:

Headers:

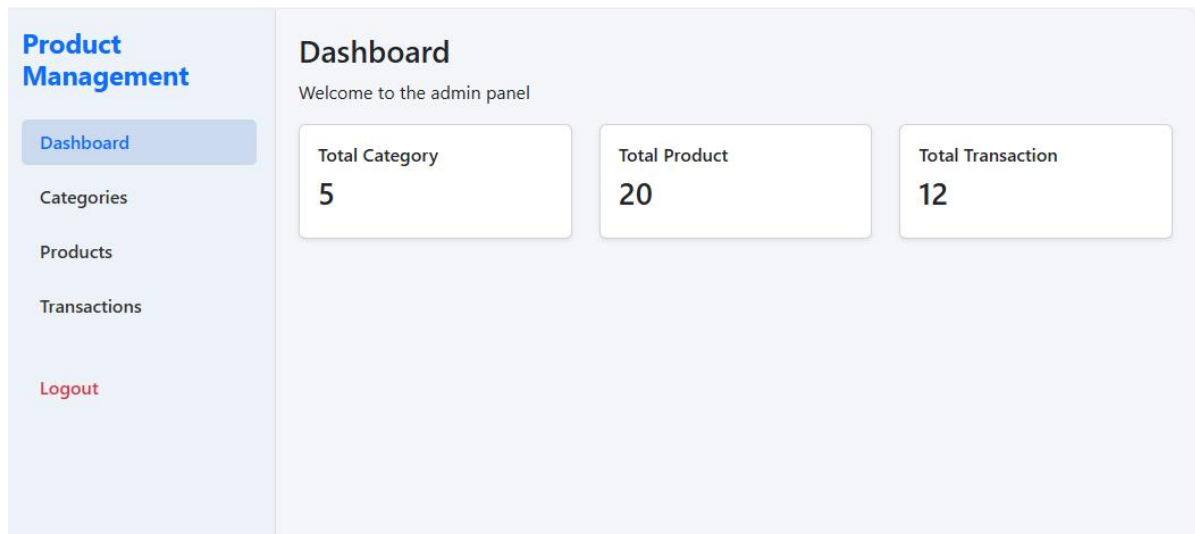
- Response status: 401

Body (JSON):

```
{
  "success": false,
  "message": "Unauthenticated."
}
```

Phase 2 : Front-end Development

In this part, you should build a front-end application using one of the provided frameworks (vue js or react js). You can use the provided template gui on the media files to build the front-end ui.



Instructions

- Save your REST Api in the directory called **“SERVER_MODULE/backend”** and save it in the repository folder.
- Save your front-end app in the directory called **“SERVER_MODULE/frontend”** and save it in the repository folder.
- When developing the Frontend, you can **use/consume the provided REST API**.
- You should **build the frontend app to production mode** and **save the production files in the “SERVER_MODULE/frontend/build” folder** before you push it to the github repository.
- Make sure the **“/build”** folder **isn't written on the .gitignore file**.
- You should commit and push your changes to the github repository at least every 30 minutes.

Scoring Rubric

Phase	Parts	Score
Back-end	Database	4
	Authentication	3
	Category	3
	Product	3
	Transaction	3
	Back-end Code Quality	4
Front-end	Authentication Implementation	3
	Category Management (CRUD UI)	3
	Product Management (CRUD UI)	4
	Transaction Management	3
	User Interface & User Experience (UI/UX)	3
	Frontend Code Quality	4
Total		40